

Introducere în programare - curs și laborator - suport electronic

(C) 2023-2024 Bogdan Pătruț

Meniu

[Lectia 1](#) | [Lectia 2](#) | [Lectia 3](#) | [Lectia 4](#) | [Lectia 5](#) | [Lectia 6](#) | [Lectia 7](#)
[Material pentru laboratoare](#)

Lectia 2 - Intrări/ieșiri, instrucțiuni și funcții

Rezumat curs 2



În acest curs se vor discuta despre funcțiile de citire și scriere (cu accent pe funcțiile `printf`, `scanf` din `stdio.h`), apoi despre instrucțiunile limbajului C, iar în final despre declararea și definirea funcțiilor în C, cu accent pe transmiterea parametrilor.

Video

- Intrări și ieșiri (8:08): [click aici](#)
- Instrucțiuni (9:30): [click aici](#)
- Exemple cu instrucțiunile `while` și `for` (15:12): [click aici](#)
- Cum gândim exercițiile cu instrucțiunea `for` (20:52): [click aici](#)
- Funcții (18:17): [click aici](#)
- Transmiterea parametrilor (13:19): [click aici](#)
- Recursivitate (10:58): [click aici](#)

2.1. Funcții de citire și scriere

În limbajul C avem două operații de citire și scriere de la tastatură (`stdin`) și la ecran (`stdout`), prin fișiere sau prin intermediul funcțiilor de bibliotecă. Pentru citirea de la tastatură și scrierea la ecran avem următoarele funcții/macrodefiniții:

- fără formatare: `getchar`, `putchar`, `getch`, `getche`, `putch`, `gets`, `puts`
- cu formatare: `scanf`, `printf`
- incluse în bibliotecile `stdio.h` (`getchar`, `putchar`, `gets`, `puts`, `scanf`, `printf`) sau `conio.h` (`getch`, `getche`, `putch`)
- Versiuni mai vechi de Code::Blocks nu includ biblioteca `conio.h`, dar se poate folosi biblioteca `conio2.h`

▼ detalii biblioteca `conio2.h`

Pentru a verifica dacă tastatură este apăsată, pentru afișare de texte cu culori și oriunde pe ecran, puteți folosi în Code::Blocks biblioteca `conio2.h`, care este o extensie a bibliotecii `conio.h`, dezvoltată în urmă cu câțiva ani de firma Borland, în produsul său Borland C++. La laborator pot fi date probleme care să utilizeze această bibliotecă. Pentru a folosi această bibliotecă, descărcați de aici fișierele necesare: <https://github.com/Fernando-Lafeta/Biblioteca-Conio-2> și instalați-o astfel:

1. obțineți fișierele `conio2.h` și `libconio.a`
2. copiați fișierul `conio2.h` în folderul „include” al compilatorului vostru MinGW, din CodeBlocks
3. copiați fișierul `libconio.a` în folderul „lib” al compilatorului vostru MinGW, din CodeBlocks
4. navigați în Code Blocks la Settings -> Compiler and debugger, apoi la „Linker settings”; la 'Link libraries' apăsați pe 'Add' și căutați calea către „libconio.a”, apoi apăsați pe 'OK'.

Iată un program demonstrativ pentru utilizarea lui `conio2.h`:

```
#include <stdio.h>
#include <conio2.h>
#define getch _getch
#define kbhit _kbhit
#define cprintf _cprintf
using namespace std;
void printAt(unsigned int x, unsigned int y, char textDeAfisat[100], unsigned int culoareText, unsigned int culoareFond)
{
    textcolor(culoareText); textbackground(culoareFond); gotoxy(x,y);
    cprintf(textDeAfisat); gotoxy(1,1);
}
int main()
{
    char tasta; unsigned int x; unsigned int y;
    x=30; y=20;
```

```

        printAt(x,y,"0",LIGHTRED, YELLOW);
    do {
        if (kbhit()) {
            printAt(x,y," ",BLACK, BLACK);
            tasta=getch();
            switch (tasta) {
                case 72: // sus
                    y--; break;
                case 80: // jos
                    y++; break;
                case 75: // stanga
                    x--; break;
                case 77: // dreapta
                    x++; break;
            }
            printAt(x,y,"0",LIGHTRED, YELLOW);
        }
    } while (tasta!=27); // escape
    textbackground(0); textcolor(7);
    return 0;
}

```

2.1.1. Funcția `printf()`

Cu ajutorul acestei funcții putem afișa mesaje pe ecran. Când programul folosește `printf`, datele pe care dorim să le afișăm reprezintă parametrii sau argumentele funcției `printf`. Exemplu: `printf("Hello!");` Funcția acceptă mai mulți parametri. Primul parametru trebuie întotdeauna să fie un șir de caractere, parametrii care urmează primului șir de caractere pot fi numere, variabile, expresii (cum ar fi `10*7`) sau chiar alte șiruri de caractere. Atunci când dorim ca să afișăm o valoare sau o variabilă, trebuie să includem informația despre tipul variabilei în primul parametru. Pe lângă caractere, în primul parametru putem să includem specificatori de format, care spun funcției cum să-i afișeze pe ceilalți parametri. De exemplu, pentru a afișa o valoare de tip `int`, vei folosi `%d`. De asemenea, pentru a tipări o valoare în virgulă mobilă (`float`), se poate folosi `%f`. Următoarele instrucțiuni ilustrează utilizarea specificatorilor de format în cadrul funcției `printf()`:

```

int varsta=18, greutate=54, pret=12;
float inaltime=1.69;
printf("Varsta este: %d\n", varsta);
//Va afișa pe ecran - Varsta este: 18
printf("Impozitul pe vanzari este: %f\n",pret*0.07);
//Va afișa pe ecran - Impozitul pe vanzari este: 0.84
printf("Varsta: %d, greutatea: %d, inaltimea: %f\n",
varsta,greutate,inaltime);

```

Va afișa pe ecran

```

Varsta: 18
Impozitul pe vanzari este: 0.840000
Varsta: 18, greutatea: 54, inaltimea: 1.690000

```

Următorul program folosește specificatorul de format `%d` pentru a afișa valori și variabile de tip `int`:

```

#include <stdio.h>
using namespace std;
int main()
{
    int varsta = 41;
    int greutate = 75;
    int inaltime = 180;
    printf("Varsta utilizatorului: %d greutate: %d inaltime: %d\n",varsta,greutate,inaltime);
    printf("%d plus %d egal %d\n",1,2,1+2);
}

```

▼ rezultat

```

Varsta utilizatorului: 41 greutate: 75 inaltime: 180
1 plus 2 egal 3

```

Exemple de utilizare

▼ click aici

▼ Tipărirea unei valori întregi octale sau hexazecimale

Funcția `printf` permite utilizarea unor specificatori de format care furnizează informații despre tipurile parametrilor (cum ar fi `int`, `float`, `char` etc.). Dacă ne dorim să afișăm o valoare întreagă octală (în baza 8) sau hexazecimală (în baza 16), vom folosi specificatorul `%o`, respectiv `%x` (pentru minuscule) și `%X` (majuscule).

```

#include <stdio.h>
int main()
{
    int valoare = 255;
    printf("Valoarea zecimala %d este %o in octal\n",
    valoare, valoare);
    printf("Valoarea zecimala %d este %x in hexazecimal\n",
    valoare, valoare);
    printf("Valoarea zecimala %d este %X in hexazecimal\n",
    valoare, valoare);
}

```

▼ rezultat

```
Valoarea zecimala 255 este 377 in octal
Valoarea zecimala 255 este ff in hexazecimal
Valoarea zecimala 255 este FF in hexazecimal
```

▼ Afișarea valorilor de tip `unsigned int`

`printf` permite utilizarea unor specificații de format care furnizează informații despre tipurile parametrilor. Pentru a afișa variabile de tip `unsigned int` trebuie să folosiți specificatorul de format `%u`. Dacă se folosește `%d` în loc de `%u`, atunci funcția `printf` va trata valoarea respectivă ca fiind de tip `int` și probabil va afișa rezultatul greșit.

```
#include <stdio.h>
int main()
{
    unsigned int valoare = 4200000000;
    printf("Afiseaza 4200000000 ca unsigned %u\n", valoare);
    printf("Afiseaza 4200000000 ca int %d\n", valoare);
}
```

▼ rezultat posibil

```
Afiseaza 4200000000 ca unsigned 4200000000
Afiseaza 4200000000 ca int -94967296
```

▼ Afișarea valorilor de tip `long int`

Pentru a afișa valorile de tip `long int` cu funcția `printf`, trebuie să folosiți specificatorul de format `%ld`. Dacă folosiți `%d` în loc de `%ld`, atunci funcția `printf` va trata valoarea respectivă ca fiind de tip `int` și probabil va afișa rezultatul greșit.

```
#include <stdio.h>
int main()
{
    long int un_milion = 1000000;
    printf("Un milion este %ld \n", un_milion);
    printf("Un milion = %d\n", un_milion);
}
```

▼ rezultat

```
Un milion este 1000000
Un milion = 16960
```

▼ Afișarea valorilor de tip `float` folosind funcția `printf`

Pentru afișarea valorilor de tip `float`, în virgulă mobilă, trebuie folosit specificatorul de format `%f`.

```
#include <stdio.h>
int main()
{
    float pret = 525.75;
    float rata_impozit = 0.06;
    printf("Pretul este %f\n", pret);
    printf("Impozitul pe vanzari este %f\n", pret*rata_impozit);
}
```

▼ rezultat

```
Pretul este 525.750000
Impozitul pe vanzari este 31.544999
```

▼ Afișarea valorilor de tip `char` folosind funcția `printf`

Pentru a afișa valorile de tip `char` trebuie utilizat specificatorul de format `%c`.

```
#include <stdio.h>
int main()
{
    printf("Litera este %c\n", 'A');
    printf("Litera este %c\n", 65);
}
```

```
}
```

▼ rezultat

```
Litera este A
Litera este A
```

▼ Afișarea valorilor în virgulă mobilă în format exponențial

În funcție de cerințele programului este posibil să afișați valori în formatul exponențial. Pentru a afișa valori în virgulă mobilă într-un format exponențial, folosiți specificatorul de format `%e` sau `%E`.

```
#include <stdio.h>
int main()
{
    float pi = 3.14159;
    float raza = 2.0031;
    printf("Aria cercului este %e\n", 2*pi*raza);
    printf("Aria cercului este %E\n", 2*pi*raza);
}
```

▼ rezultat

```
Aria cercului este 1.258584e+01 Aria cercului este 1.258584E+01
```

▼ Afișarea valorilor în virgulă mobilă

Cu specificatorul `%f` putem indica lui `printf` să afișeze valori în virgulă mobilă în format zecimal. De asemenea, putem folosi specificatorii `%e`, `%E` pentru a afișa valori în virgulă mobilă în format exponențial. În mod similar, funcția `printf()` acceptă specificatorii de format `%g`, `%G`. Atunci când folosim acești specificatori, `printf` alege formatul `%f` sau `%e`, astfel încât să ofere utilizatorului o afișare cât mai convenabilă. Următorul program ilustrează utilizarea specificatorului de format `%g`:

```
#include <stdio.h>
int main()
{
    printf("Numarul 0.1234 este afisat in formatul %g\n", 0.1234);
    printf("Numarul 0.00001234 este afisat in formatul %g\n", 0.00001234);
}
```

▼ rezultat

```
Numarul 0.1234 este afisat in formatul 0.1234 Numarul 0.00001234 este afisat in formatul 1.234e-05
```

▼ Afișarea unui șir de caractere folosind funcția printf

Un șir de caractere este o secvență care conține zero sau mai multe caractere. Una dintre cele mai obișnuite operații executate în programe este afișarea șirurilor de caractere. Așa cum ați învățat pe parcursul acestui articol, funcția `printf` permite utilizarea unor specificatori de format care furnizează informații despre tipurile parametrilor (cum ar fi `int`, `float`, `char` etc). Pentru a afișa un șir de caractere cu funcția `printf` trebuie să folosiți specificatorul de format `%s`. Următorul program utilizează acest specificator pentru a afișa un șir de caractere:

```
#include <stdio.h>
int main()
{
    char titlu[255] = "Funcțiile printf() și scanf()";
    printf("Numele acestui articol este: %s\n", titlu);
}
```

▼ rezultat

```
Numele acestui articol este: Funcțiile printf() și scanf()
```

▼ Utilizarea semnului plus sau minus înaintea unei valori

Atunci când folosiți funcția `printf` pentru a afișa o valoare negativă, valoarea va fi întotdeauna precedată de semnul minus. În funcție de programul dumneavoastră, vă ați dori uneori ca `printf` să afișeze și semnul pentru valorile pozitive. Pentru a indica funcției `printf` să afișeze semnul valorii, pur și simplu veți include un semn plus imediat după `%` în specificatorul de format. Următorul program, ilustrează utilizarea semnului plus pentru specificatorii de format:

```
#include <stdio.h>
int main()
```

```

{
    int neg_int = -5;
    int poz_int = 5;
    float neg_float = -100.23;
    float poz_float = 100.23;
    printf("Valoarea intreaga este %d si %d\n", neg_int, poz_int);
    printf("Valoarea in virgula mobila este %f si %f\n", neg_float, poz_float);
}

```

▼ rezultat

```

Valoarea intreaga este -5 si +5
Valoarea in virgula mobila este -100.230003 si +100.230003

```

▼ Formatarea unei valori intregi folosind functia printf

Daca folositi specificatorul de format %d, puteti sa indicati functiei printf sa afiseze un numar minim de caractere. Urmatorul program ilustreaza modul in care puteti formata valorile intregi folosind specificatorul %d:

```

#include <stdio.h>
int main()
{
    int valoare1=332, valoare2=12000, valoare3 = 3311;
    printf("%7d\n", valoare1);
    printf("%7d\n", valoare2);
    printf("%7d\n", valoare3);
}

```

▼ rezultat

```

    332
  12000
   3311

```

Cifra pe care o plasati dupa % precizeaza numarul minim de caractere pe care printf il foloseste pentru a afisa o valoare intreaga. Daca, de exemplu specificati %5d si valoarea pe care vreti sa o afisati este 10, printf va introduce trei spatii libere inainte de valoare. Retineti ca valoarea precizeaza numarul minim de caractere pe care il va avea iesirea. Daca valoarea pe care doriti sa o afisati are mai multe caractere decat ati precizat, printf va folosi numarul de caractere necesar pentru afisarea corecta a valorii.

▼ Afisarea numerelor intregi precedate de zero

În funcție de scopul programului, poate dorm ca să afișăm zerouri înaintea valorii, în loc de spații. Pentru a indica funcției `printf` să adauge zerouri înaintea valorii, se plasează un 0 (zero nu litera "O") imediat dupa % în specificatorul de format, înainte de numărul de cifre dorit. Următorul program ilustrează acest procedeu:

```

#include <stdio.h>
int main()
{
    int valoare = 5;
    printf("%01d\n", valoare);
    printf("%02d\n", valoare);
    printf("%03d\n", valoare);
    printf("%04d\n", valoare);
}

```

▼ rezultat

```

5
05
005
0005

```

▼ Afișarea unui prefix înaintea valorilor octale și hexazecimale

Pentru a indica funcției să adauge prefixul potrivit unei valori octale sau hexazecimale, se plasează caracterul # imediat dupa % în specificatorul de format.

```

#include <stdio.h>
int main()
{
    int valoare = 255;
    printf("Valoarea zecimala %d este %#o in octal\n", valoare, valoare);
    printf("Valoarea zecimala %d este %#x in hexazecimal\n", valoare, valoare);
    printf("Valoarea zecimala %d este %#X in hexazecimal\n", valoare, valoare);
}

```

▼ rezultat

```
Valoarea zecimala 255 este 0377 in octal
Valoarea zecimala 255 este 0xff in hexazecimal
Valoarea zecimala 255 este 0xFF in hexazecimal
```

▼ Formatarea valorilor cu virgulă mobilă

Atunci când formatăm o valoare cu virgulă mobilă, trebuie să specificăm două valori. Prima precizează numărul de caractere pe care vrem să-l afișăm, iar a doua valoare reprezintă numărul de cifre pe care dorim să le afișăm la dreapta punctului zecimal.

```
#include <stdio.h>
int main()
{
    float valoare = 1.23456;
    printf("%8.1f\n", valoare);
    printf("%8.3f\n", valoare);
    printf("%8.5f\n", valoare);
}
```

În mod similar putem cere funcției să formateze o valoare în virgulă mobilă, în format exponențial.

▼ Alinierea la stânga a ieșirii

Atunci când se afișează un text, utilizând formatarea cu `printf`, alinierea prestabilită este la dreapta. Dacă dorim ca să facem alinierea textului la stânga, trebuie să plasăm un semn minus (-) imediat după % în specificatorul de format.

```
#include <stdio.h>
int main()
{
    int val_int = 5; float val_flt = 3.33;
    printf("Aliniere dreapta %5d a valorii \n", val_int);
    printf("Aliniere stanga %-5d a valorii\n", val_int);
    printf("Aliniere dreapta %7.2f a valorii\n", val_flt);
    printf("Aliniere stanga %-7.2f a valorii\n", val_flt);
}
```

▼ rezultat

```
Aliniere dreapta      5 a valorii
Aliniere stanga 5    a valorii
Aliniere dreapta      3.33 a valorii
Aliniere stanga 3.33  a valorii
```

▼ Utilizarea combinată a specificatorilor de format

Cu ajutorul funcției `printf` putem utiliza specificatori de format combinați, de exemplu afișarea la stânga a unui număr întreg cu semn:

```
#include <stdio.h>
int main()
{
    int val_int = 5;
    printf("Afisarea la stanga cu semn %+-3d\n",val_int);
}
```

▼ rezultat

```
Afisarea la stanga cu semn +5
```

2.1.2. Funcția `scanf()`

Funcția `scanf` este o funcție cu ajutorul căreia putem citi date de la tastatură. Sintaxa funcției `scanf` este următoarea:

```
scanf("lista de formate",adresa_var1, adresa_var2, ...);
```

Această funcție realizează următoarele operații:

- citește din fișierul standard de intrare `stdio` o secvență de câmpuri de intrare, caracter cu caracter, pâna la terminarea introducerii câmpurilor și apăsarea tastei `<enter>`;
- formatează fiecare câmp conform formatului specificat în lista de formate; Din caracterele citite se calculează valori numerice sau literale, conform tipului fiecărei variabile, dimensiunile de format specificate și a separatorilor de câmpuri predefiniți (spațiu, `<tab>` și `<enter>`) sau impuși explicit
- valorile astfel construite sunt stocate la adresele variabilelor specificate ca argumente.

Indiferent de formatul folosit, la întâlnirea unui spațiu în introducerea datelor, este terminată citirea variabilei. Pentru funcția de citire `scanf` trebuie folosit operatorul de adresă `&`. Pentru variabilele citite cu această funcție trebuie precizate adresele la care se stochează în memoria calculatorului valorile variabilelor. Funcția va introduce

valorile citite direct la acele adrese. Singurul caz în care nu este obligatorie folosirea operatorului adresă pentru citirea valorii unei variabile cu funcția `scanf` este citirea unui șir de caractere.

Observație: Citirea cu ajutorul funcției `scanf()` a șirurilor de caractere care conțin spații este imposibilă. În cazul în care formatul specificat este necorespunzător, rezultatul obținut poate fi neprevăzut. Valoarea întoarsă de `scanf` în caz de succes, este numărul de variabile care au fost citit corect. Dacă nu a fost citită nicio variabilă (de exemplu s-a introdus un șir în loc de un număr) funcția întoarce valoarea 0. Dacă apare o eroare înaintea oricărei citiri și asignări, funcția returnează EOF (constantă de sistem având valoarea întregă -1).

▼ Specificatorii de format ai funcției `scanf`

Specificatorii de format ai funcției `scanf` sunt prezenți în lista următoare:

- `%c` – citește un caracter
- `%d` – citește un întreg zecimal
- `%i` – citește un întreg zecimal
- `%e` – citește un număr float
- `%f` – citește un număr float
- `%g` – citește un număr float
- `%o` – citește un număr octal fără semn
- `%s` – citește un șir de caractere
- `%x` – citește un nr hexazecimal fara semn
- `%p` – citește un pointer
- `%n` – argumentul asociat primește o valoare întreaga egală cu numărul de caractere deja citite
- `%u` – citește un număr întreg fără semn
- `[%]` – scanare pentru set de caractere

Observație: Multe compilatoare tratează specificatorul de format `%i` ca fiind identic cu `%d`. Totuși nu este recomandat să îl folosiți deoarece `%i` este un specificator mostenit și este posibil ca viitoarele compilatoare să nu îl accepte.

În specificatorul de format pot să apară și modificatori de tip:

- modificatorul `h`, care poate precede caracterele de conversie `d, i, o, u, x, X`, precizează că valoarea convertită trebuie memorată ca un `short int` sau `unsigned short int`;
- modificatorul `l`, care poate precede caracterele de conversie `d, i, o, u, x, X`, caz în care valoarea trebuie memorată ca un `long int` sau `unsigned long int`, sau poate precede caracterele `e, E, f, g, G`, caz în care valoarea trebuie memorată ca un `double`;
- modificatorul `L`, care poate precede numai caracterele `e, E, f, g, G` și precizează că valoarea convertită trebuie memorată ca un `long double`.

Un număr între `%` și caracterul de format limitează caracterele citite: de exemplu, `%4d` reprezintă un întreg din cel mult 4 caractere (spațiile inițiale nu contează).

```
scanf("%d%d", &m, &n); 12 34 m=12 n=34
scanf("%2d%2d", &m, &n); 12345 m=12 n=34 rest: 5
scanf("%d.%d", &m, &n); 12.34 m=12 n=34
scanf("%f", &x); 12.34 x=12.34
scanf("%d%x", &m, &n); 123a m=123 n=0xA
```

2.1.3. Funcțiile `getchar` și `putchar`

▼ Se folosesc pentru operații de citire și scriere a caracterelor.

- `int getchar(void)` - citește un caracter de la tastatură. Așteaptă până este apasată o tastă și returnează valoarea sa; tasta apăsată are imediat ecou pe ecran.
- `int putchar(int c)` - scrie un caracter pe ecran în poziția curentă a cursorului
- fișierul antet pentru aceste funcții este `stdio.h`.

2.1.4. Funcțiile `gets` și `puts`

▼ Se folosesc pentru operații de citire și scriere a șirurilor de caractere.

- `char *gets(char *s)` – citește caractere din `stdin` și le depune în zona de date de la adresa `s`, până la apăsarea tastei <Enter>. În șir, tastei <Enter> îi va corespunde caracterul `'\0'`.
- dacă operația de citire reușește, funcția întoarce adresa șirului, altfel valoarea `NULL` (`= 0`).
- `int puts(const char *s)` – scrie pe ecran șirul de la adresa `s` sau o constantă șir de caractere și apoi trece la linie nouă.
- dacă operația de scriere reușește, funcția întoarce ultimul caracter, altfel valoarea `EOF` (-1).
- fișierul antet pentru aceste funcții este `stdio.h`.

Nu recomandăm folosirea funcției `gets`, deoarece:

- `char *gets(char *s)` - primește ca intrare numai un buffer (`s`), nu știm dimensiunea lui
- poate apărea problema de *buffer overflow*: citim în `s` mai mult decât dimensiunea lui, deoarece `gets` nu ne împiedică, așa că va scrie datele în altă parte

În schimb, se poate folosi funcția `fgets` cu fișierul `stdin` ca fișier de intrare: `char *fgets(char *s, int size, FILE *stream): fgets(s, sizeof(s), stdin)`. De altfel, în standardul C11 funcția `gets` este eliminată

2.1.5. Intrări / ieșiri folosind stream-uri (cu `cin/cout`) în C++

Stream-urile au în principal rolul de a abstractiza operațiile de intrare-ieșire. Ele oferă metode de scriere și citire a datelor independente de dispozitivul I/O și chiar independente de platformă. Stream-urile ascund problemele specifice dispozitivului cu care se lucrează, folosind biblioteca standard `iostream`. Această bibliotecă utilizează un sistem de zone tampon (buffer). Operațiile de intrare și ieșire cu dispozitivele periferice sunt consumatoare de timp, de aceea aplicațiile sunt uneori nevoite să aștepte terminarea acestor operații. Datele trimise unui stream nu sunt scrise imediat pe dispozitivul respectiv, ci sunt transferate într-o zonă de memorie tampon, din care sunt descărcate către dispozitiv abia în momentul umplerii acestei zone. În C++ stream-urile au fost implementate utilizând clase, după cum urmează: clasa `streambuf` gestionează buffer-ele. clasa `ios` este clasa de bază pentru clasele de stream-uri de intrare și de ieșire. Clasa `ios` are ca variabilă membru un obiect de tip `streambuf`. clasele `istream` și `ostream` sunt derivate din `ios`. clasa `iostream` este derivată din `istream` și `ostream` și oferă metode pentru lucrul cu terminalul. clasa `fstream` oferă metode pentru operații cu fișiere. Când un program C++ care include `iostream.h` este lansat în execuție, sunt create și inițializate automat patru obiecte:

- `cin` gestionează citirile de la intrarea standard (tastatură);
- `cout` gestionează scrierile către ieșirea standard (ecranul);
- `cerr` gestionează ieșirea către dispozitivul standard de eroare (ecranul), neutilizând zone tampon;
- `clog` gestionează ieșirea către dispozitivul standard de eroare (ecranul), utilizand buffere.

Dispozitivele standard de intrare, ieșire și eroare pot fi redirectate către alte dispozitive. Erorile sunt de obicei redirectate către fișiere, iar intrarea și ieșirea pot fi trimise către fișiere utilizând comenzi ale sistemului de operare (utilizarea ieșirii unui program ca intrare pentru altul). Forma generală de utilizare a lui `cin` și `cout` este: `cin >> var;` /* citește var de la cin */ Se pot prelua tipurile aritmetice, șiruri de caractere `cout << expr;` /* scrie expr la cout */ Se pot transfera tipurile aritmetice, șiruri de caractere, pointeri de orice tip în afară de char. Sunt posibile operații multiple, de tipul: `cin >> var1 >> var2 ... >> varN;` și `cout << var1 << var2 ... << varN;`

▼ Exemplu - citirea și afișarea unui `int`

```
cin >> variabile      cout << expresii
```

•citirea unui int

```
std::cout << "introduceți valoarea lui a: ";
std::cin >> a;
```

•afișarea unui int

```
int a = 10;
std::cout << a;
```

▼ Exemplu - citirea și afișarea unui `char`

```
/* Exemple de constante caracter */
#include <iostream>
using namespace std;
int main()
{
    char a, b, c, d;
    a = 'A'; b = 65; c = '\101'; d = '\x41';
    cout << a << b << c << d << endl;
    cout << a << (int)a;
    cout << oct << (int)a << hex << (int)a;
    return 0;
}
```

▼ Exemplu - afișarea codurilor ASCII

```
#include <iostream>
using namespace std;
int main ()
{
    short c;
    for(c = 0; c <= 127; c++){ // for(c='a'; c<='z'; c++)
        cout << "cod ASCII: " << (int)c;
        cout << ", caracter: ";
        cout << (char)c << endl;
    }
    return 0;
}
```

▼ Explicații

Așa cum veți învăța la disciplina *Programarea orientată pe obiecte*, de fapt, `cin` și `cout` sunt niște obiecte definite global, care au *supraincărcat* operatorii `>>`, `<<` de mai multe ori, pentru fiecare tip de parametru în parte (`int`, `char` * etc.):

`istream &operator >> (TipParametru &)`

De exemplu:

```
#include <iostream.h>
void main()
{
    int IntegerNumber;
    cout << "IntegerNumber = ";
    cin >> IntegerNumber;
    cout << "\nWhat you entered = " << IntegerNumber << endl;
}
```

Aici am citit de la intrarea standard o valoare întreagă, ce s-a trimis apoi ieșirii standard. Simbolurile `'\n'`, `'\t'`, ș.a.m.d. se pot utiliza ca la funcțiile `printf`, `scanf` etc. Utilizarea simbolului `endl` va forța golirea zonei tampon, adică trimiterea datelor imediat către ieșire. Atât operatorul `>>`, cât și `<<` returnează o referință către un obiect al clasei `istream`. Deoarece `cin`, respectiv `cout` este și el un obiect `istream`, valoarea returnată de o operație de citire/scriere din/în stream poate fi utilizată ca intrare/ieșire pentru următoarea operație de același fel.

Obiectul `cin` are o funcție membru `get`, ce poate fi utilizată pentru a obține un singur caracter din intrare, apelând-o fără niciun parametru, caz în care returnează valoarea utilizată, sau ca referință la un caracter. În forma `get()` fără parametri, funcția întoarce valoarea caracterului găsit. Spre deosebire de operatorul `>>`, nu poate fi utilizată pentru a citi mai multe intrări, deoarece valoarea returnată este de tip întreg, nu un obiect `istream`.

▼ Exemplu

```
#include <iostream.h>
void main()
{
    char c;
    while((c = cin.get()) != EOF)
    {
        cout << "c = " << c << endl;
    }
}
```


Operatorul `>>` nu poate fi utilizat pentru a citi corect șiruri de caractere de la intrare (cu spații), deoarece spațiile sunt interpretate ca separator între diverse valori de intrare. În astfel de cazuri trebuie folosită funcția `get()`. În acest caz, funcția `get` se folosește astfel:

```
cin.get(char *PointerLaSirulDeCaractere, int LungimeMaxima, char Sfarsit);
```

Primul parametru este un pointer la zona de memorie în care va fi depus șirul de caractere. Al doilea parametru reprezintă numărul maxim de caractere ce poate fi citit plus unu. Cel de-al treilea parametru este caracterul de încheiere a citirii, care este opțional (implicit considerat ca fiind `'\n'`). În cazul în care caracterul de încheiere este întâlnit înainte de a fi citit numărul maxim de caractere, acest caracter nu va fi extras din stream.

Există o funcție similară funcției `get()`, cu aceeași sintaxă, numită `getline()`. Funcționarea sa este identică cu cea a funcției `get()`, cu excepția faptului că acel ultim caracter menționat mai sus este și el extras din stream.

▼ Alte funcții ale lui `cin`

- Funcția `cin.ignore()` se utilizează pentru a trece peste un număr de caractere, până la întâlnirea unui anumit caracter: `cin.ignore(int NumarMaximDeCaractereIgonorate, char Sfarsit);`.
- Funcția `cin.peek()` returnează următorul caracter din stream, fără însă a-l extrage.
- Funcția `cin.putback()` inserează în stream un caracter.

Obiectul `cout` are o funcție `cout.flush()` ce determină trimiterea către ieșire a tuturor informațiilor aflate în zona de memorie tampon. Această funcție poate fi apelată și sub forma `cout << flush`. Funcția `cout.put()` scrie un caracter către ieșire: `cout.put(char Caracter);`. Deoarece această funcție returnează o referință de tip `ostream`, pot fi utilizate apeluri succesive ale acesteia, de exemplu: `cout.put('H').put('i').put('!').put('\n');`.

Funcția `cout.write()` are același rol ca și operatorul `<<`, cu excepția faptului că se poate specifica numărul maxim de caractere ce se doresc scrise. Sintaxa funcției este: `cout.write(char *SirDeCaractere, int CaractereDeScris);`.

Funcția `cout.width()` permite modificarea dimensiunii valorii trimise spre ieșire (pentru următoarea operație), care implicit este considerată exact mărimea câmpului în cauză: `cout.width(int Dimensiune);`. Funcția `cout.fill()` permite modificarea caracterului utilizat pentru umplerea eventualului spațiu liber creat prin utilizarea unei dimensiuni mai mari decât cea necesară ieșirii, cu funcția `cout.width():cout.fill(char Caracter);`.

▼ Opțiuni de formatare a ieșirii

Pentru formatarea ieșirii sunt definite două funcții membre ale `cout`, și anume:

- Funcția `cout.setf()` activează o opțiune de formatare a ieșirii, primită ca parametru: `cout.setf(ios::Optiune);`, unde `Optiune` poate fi:
 - `showpos` - determină adăugarea semnului plus (+) în fața valorilor numerice pozitive;
 - `left`, `right`, `internal` - schimbă alinierea ieșirii;
 - `dec`, `oct`, `hex` - schimbă baza de numerație pentru valori numerice;
 - `showbase` - determină adăugarea identificatorului bazei de numerație în fața valorilor numerice.
- Funcția `cout.setw()` modifică dimensiunea ieșirii, fiind similară funcției `cout.width():cout.setw(int Dimensiune);`

▼ Exemplu

```
#include <iostream.h>
#include <iomanip.h>
void main()
{
    int number = 783;
    cout << "number = " << number;
    cout.setf(ios::showbase);
    cout << "number in hexa = " << hex << number;
    cout.setf(ios::left);
    cout << "number in octal, aligned to the left = " << oct << number;
}
```

2.2. Instrucțiuni

În limbajul C avem următoarele tipuri de instrucțiuni:

- Expresii
- Instrucțiuni compuse (bloc)
- Instrucțiuni condiționale
- Instrucțiuni iterative
- Instrucțiuni pentru oprirea secvenței
- Instrucțiunea de salt necondiționat

▼ Detalii

- Expresii: `expresie; expresie;`
- Compuse (bloc): `{declarații instrucțiuni}`
- Condiționale: `if if-elseswitch-case`
- Iterative: `for while do-while`
- Întreruperea secvenței: `continue; break; return expr;`
- Salt necondiționat: `goto`

2.2.1. Instrucțiunea expresie

▼ Sintaxa

```
instr_expresie ::= { expresie }opt ;
```

▼ Semantica

1. Se evaluează expresia.
2. Dacă este o expresie de forma unei instrucțiuni de atribuire, **variabila=expresie**, atunci **variabila** primește valoarea **expresiei** din dreapta, vechea valoare a expresiei pierzându-se.
3. Dacă este o expresie de forma **variabila op = expresie**, aceasta este echivalentă cu **variabila=variabila op expresie**, unde **op** este un operator din mulțimea {+, -, *, /, %}
4. Dacă este o expresie de forma **variabila++** sau **++variabila**, aceasta este echivalentă cu **variabila=variabila+1**.
5. Dacă este o expresie de forma **variabila--** sau **--variabila**, aceasta este echivalentă cu **variabila=variabila-1**.

▼ Exemple

```
a = b;
a + b + c;
;
cout << a;
sizeof(int);
```

2.2.2. Instrucțiunea compusă (bloc)

▼ Sintaxa

Grupează instrucțiuni/declarații într-o unitate executabilă. O instrucțiune compusă este ea însăși o instrucțiune: oriunde poate să apară o instrucțiune, este corect să apară și o instrucțiune compusă.

instr_compusa ::= { lista_de_declarații | lista_de_instrucțiuni }₀₊

▼ Semantica

1. Se execută o singură dată fiecare dintre instrucțiuni/declarații, de sus în jos și de la stânga la dreapta.
2. Nu se revine în vreun fel la una dintre instrucțiunile/declarațiile anterioare.

▼ Exemple

```
{
    int a=3, b=10, c=7;
    a += b += c;
    cout << a << " ", " << b << " ", " << c; // ?, ?, ?
}
if (x > y){
    int temp;
    temp = x; x = y; y = temp;
    cout << x << y;
}
{
    int a, b, c;
    {
        b = 2; c = 3; a = b += c;
    }
    cout << "a= " << a <
```

2.2.3. Instrucțiunile condiționale **if** și **if-else**

▼ Sintaxa

```
instr_if ::= if (<expresie_booleană>)
                                     {<instrucțiune1>;}
instr_if-else ::= if (<expresie_booleană>)
                                     {<instrucțiune1>;}
                                     else
                                     {<instrucțiune2>;}
```

expresie_booleană este construită cu:

- Expresii aritmetice
- Comparatori: ==, !=, <, <=, >, >=
- Conectori logici: &&, ||, !

▼ Semantica

1. Se evaluează (o singură dată) **expresie_booleană**.
2. Dacă aceasta este adevărată, atunci se execută **instrucțiune1**.
3. Dacă nu este adevărată, atunci, dacă este prezentă ramura cu **else**, se execută **instrucțiune2**.

▼ Exemple

```
if(lungime == latime)
    ariaPatrat = lungime*lungime;
if (a % 2) if (b % 2) p = 1; else p = 2; // cu cine este grupat acest „else”?
if(a%2) { if(b%2) p = 1;} else p = 2;
if (primulNumar < alDoileaNumar)
    minimul = primulNumar;
else
    minimul = alDoileaNumar;

int primul, alDoilea, alTreilea, alPatrulea, maximul;
if(primul>alDoilea)
    if(alTreilea>alPatrulea)
```

```

        if(primul>alTreilea) maximul = primul;
        else maximul = alTreilea;
    else
        if(primul>alPatrulea) maximul = primul;
        else maximul = alPatrulea;
    else
        if(alTreilea>alPatrulea)
            if(alDoilea>alTreilea) maximul = alDoilea;
            else maximul = alTreilea;
        else
            if(alDoilea>alPatrulea) maximul = alDoilea;
            else maximul = alPatrulea;

```

Problema "dangling else" Regula este: **else** este atașat celui mai apropiat **if**. if (a == 1) if (b == 2) // b=2 cout << "*****\n"; else cout << "ooooo\n";

- Nu lăsați forma codului să vă ducă în eroare!
- Atenție la diferența dintre operatorii de egalitate și cel de asignare

```

if ( conditie-1 ) {
    instructiuni-1;
}
else if ( conditie-2 ) {
    instructiuni-2;
    ...
}
else if ( conditie-n ) {
    instructiuni-n;
}
else {
    instructiuni-pt-restul-posibilitatilor;
}

```

▼ Exemplu cu un calculator simplu

```

int main(void){
    float operand1, operand2, rezultat;
    char op;
    cout << "Expresia:(numar operator numar - FARA SPATII)\n";
    cin >> operand1 >> operator >> operand2;
    if(op == '+')
        rezultat = operand1+operand2;
    else if(op == '-')
        rezultat = operand1-operand2;
    else if(op == '*')
        rezultat = operand1*operand2;
    else if(op == '/')
        rezultat = operand1/operand2;
    else{
        cout << "Eroare in scrierea expresiei!";
    }
    return 1;
}
cout << "Rezultatul este: " << rezultat << "\n";
return 0;
}

```

2.2.4. Instrucțiunea **switch**

▼ Sintaxă

```

switch (expresia)
{
    case constanta1:
        grupul_de_instructiuni_1; [break;] (break este opțional)
    case constanta2:
        grupul_de_instructiuni_2; [break;]
    ...
    [default:
        grup_implicit_de_instructiuni] (default este opțional)
}

```

- Valorile constantelor trebuie să fie diferite; ordinea lor nu are importanță.
- Acoladele ce grupează mulțimea **case**-urilor sunt obligatorii.
- După fiecare **case** pot apărea mai multe instrucțiuni fără a fi grupate în acolade.

▼ Semantica:

- Se evaluează **expresia**.
- Dacă ea are valoarea **constant1**, atunci se execută **grupul_de_instructiuni_1**, eventual **grupul_de_instructiuni_2** etc., până la întâlnirea primului **break**.
- Dacă ea are valoarea **constant2**, atunci se execută **grupul_de_instructiuni_2** etc., până la întâlnirea primului **break**.
- ...
- În cazul în care **expresia** nu este egală cu niciuna din variantele **constant1**, **constant2** etc, iar **grupul_implicit_de_instructiuni** este prezent, se execută acesta. (acesta trebuie să apară o singură dată, nu neapărat la sfârșit)

▼ Exemple:

1) Echivalență între `switch` și `if`:

```
switch (x) {                if (x == 1)
  case 1:                    {
    cout << "x is 1";        cout << "x is 1";
    break;                   }
  case 2:                    else if (x == 2) {
    cout << "x is 2";        cout << "x is 2";
    break;                   }
  default:                   else {
    cout << "value of x unknown"; cout << "value of x unknown";
  }                          }

cin >> i;
switch(i){
  case 1: cout << " 1";
  case 2: cout << " 2";
  case 3: cout << " 3"; // break; // ???
  case 4: cout << " 4";
  default: cout << " blabla! ";
}

//3
```

2) Un alt exemplu

```
3
3 4 blabla !
```

3) Încă un exemplu

```
char eval;
cin >> eval;
switch (eval) {
  case 'A':
  case 'a':
    cout << "Excellent: you got an \"A\"!\n";
    break;
  case 'B':
  case 'b':
    cout << "Good: you got a \"B\"!\n";
    break;
}

b
Good: you got a "B"!
```

2.2.5. Instrucțiunea `while`

▼ Sintaxa:

```
instrucțiunea_while ::=
  while(expresie) instrucțiune

  while (expresie){
    instrucțiune
  }
  instrucțiunea_urmatoare
```

▼ Semantica:

1. Se evaluează `expresia` (condiția).
2. Dacă ea este nulă / nu este adevărată, se trece la `instrucțiunea_următoare`.
3. Dacă este adevărată, se execută `instrucțiunea`, apoi se reia ciclul, testându-se din nou valoarea de adevăr (`!=0`) a `expresiei` ș.a.m.d.
4. Așadar, cât timp `expresia` este adevărată, se execută `instrucțiunea`.

▼ Exemplu:

```
int n, i=1, factorial=1;
cin>>n;
while (i++ < n)
  factorial *= i;
cout<<factorial;
```

2.2.6. Instrucțiunea `do-while`

▼ Sintaxa

```
instrucțiunea_do..while ::=
  do instrucțiune while(conditie);

  do{
    instrucțiune
  } while (conditie);
  instrucțiunea_urmatoare
```

▼ Semantica:

1. Se execută `instrucțiune`.
2. Se evaluează `conditie`, apoi:

- o dacă valoarea sa este nenulă controlul este transferat înapoi, la începutul instrucțiunii `do..while`;
- o dacă valoarea este nulă se execută `instrucțiunea_urmatoare`.

Așadar `instrucțiune` se execută o dată sau de mai multe ori.

▼ Exemplu

```
unsigned long n;
do {
    cout << "Enter number (0 to end): ";
    cin >> n;
    cout << "You entered: " << n << "\n";
} while (n != 0);
return 0;
```

```
Enter number (0 to end): 25
You entered: 25
Enter number (0 to end): 36
You entered: 36
Enter number (0 to end): 0
You entered: 0
```

▼ Exemplu cu un calculator simplu

```
int main(void){
float operand1, operand2, rezultat;
char op, raspuns;
int ERROR;
cout << "Calculator pentru expresii de forma \n operand1 operator operand2\n";
cout << "Folositi operatorii + - * / \n";
do{
    ERROR = 0;
    cout << "Dati expresia: ";
    cin >> operand1 >> operator >> operand2;
    switch(op){
        case '+': rezultat = operand1+operand2; break;
        case '-': rezultat = operand1-operand2; break;
        case '*': rezultat = operand1*operand2; break;
        case '/': if(operand2 != 0) rezultat = operand1/operand2;
                else {cout << "Impartire prin zero!\n"; ERROR = 1;}
                break;
        default : {cout << "Operator necunoscut!\n"; ERROR = 1;}
    }
    if(!ERROR)
        cout << operand1 << " " << operator << " " << operand2 << " = " << rezultat;
    cin.sync();
    do{cout << "\n Continuati (d/n)?"; raspuns = getchar();
    } while (raspuns != 'd' && raspuns != 'n');
    } while (raspuns != 'n');
    cout << "La revedere!\n";
    return 0;
}
```

2.2.7. Instrucțiunea for

▼ Sintaxa

`instrucțiunea_for ::=`

```
for (expr-initial; expr-cond; expr-in/decrementare)
instrucțiune;
```

```
for (expr1; expr2; expr3){
    instrucțiuni;
}
instrucțiunea_urmatoare
```

Una, două sau chiar toate cele trei expresii pot lipsi, dar cei doi separatori (;) sunt obligatorii.

▼ Semantica

1. Dacă `instrucțiune` nu conține `continue` și `expr-cond` este prezentă, atunci `for` este echivalent cu:

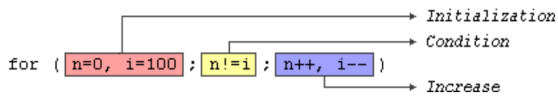
```
expr-init;
while(expr-cond){
    instrucțiune;
    expr-in/decrementare;
}
instrucțiunea_urmatoare
```

2. Dacă există `continue` atunci această instrucțiune transferă controlul la `expr-in/decrementare`.
3. Se evaluează `expr-init` - în general aceasta se utilizează pentru inițializarea iterației.
4. Se evaluează `expr-cond` - în general aceasta este o expresie logică ce se utilizează pentru controlul iterației. Dacă valoarea sa este nenulă (`true`), se execută corpul buclei (`instrucțiune`), se evaluează `expr-in/decrementare` și controlul este trecut la începutul buclei, fără a se mai evalua `expr-init`.
5. În general `expr-in/decrementare` face trecerea la iterația următoare: modifică o variabilă ce intră în componența lui `expr-cond`.
6. Procesul continuă până când valoarea `expr-cond` este nulă (`false`). Controlul este transferat următoarei instrucțiuni (cea de după `for`).

▼ Exemple

Exemplul 1:

```
int n;
cin>>n;
for (; n>0; n--) {
    cout << n << ", ";
}
cout << "GATA!\n";
```



Exemplul 2:

```
i = 1;
suma = 0;
for(;i <= N; ++i) suma += i;
```

Exemplul 3:

```
i = 1;
suma = 0;
for(;i <= N;) suma += i++;
```

Exemplul 3:

```
i = 1;
suma = 0;
for(;;) suma += i++; // Bucla infinita
```

2.2.8. Instrucțiuni de întrerupere a secvenței

1. **break;**
 - o se referă la bucla sau instrucțiunea switch cea mai apropiată.
 - o produce ieșirea din buclă sau din switch și trece controlul la instrucțiunea următoare
2. **continue;**
 - o se referă la bucla (for, while, do..while) cea mai apropiată.
 - o întrerupe execuția iterației curente și trece controlul la iterația următoare.
3. **goto;**
 - o Permite saltul la o anumită secțiune din program, identificată prin punctul de începere.
4. **return expr;** sau **return;**

Exemple:

```
int n;
for (n=10; n>0; n--)
{
    cout << n << ", ";
    if (n==3)
    {
        cout << "countdown aborted!";
        break;
    }
}

for (int n=10; n>0; n--) {
    if (n==5) continue;
    cout << n << ", ";
}
cout << "FIRE!\n";
int n=10;
loop:
cout << n << ", ";
n--;
if (n>0) goto loop;
cout << "FIRE!\n";
return 0;

int i, suma=0;
for(i = 1; i<=N; i++){
    if(i%3 != 0) continue;
    suma+=i;
}
cout << "suma = " << suma; /* suma multiplilor de 3 până la N */
```

2.2.6. Instrucțiuni de iterație - recomandare

Pentru expresia de control a iterației se recomandă operatorii relaționali în locul celor de (ne)egalitate:

```
int main(){
    int contor = 0;
```

```

double suma = 0.0, x;
for(x = 0.0; x != 9.9; x += 0.1){
    suma += x;
    ++contor;
}
cout << "suma = " << suma << ", contor = " << contor << "\n";
return 0;
}

```

suma = 495, contor = 100

Folosiți `x < 9.9` în loc de `x != 9.9`!

2.3. Definirea și apelul funcțiilor

2.3.1. Definirea unei funcții

```

tip_returnat  nume_funcție(tip1 var1,...){
    lista_de_declarații
    lista_de_instrucțiuni
}

```

2.3.2. Apelul unei funcții

`nume_funcție(expr1,...)`

Argumente

Argumentele sunt expresii ce substituie parametrii la un apel: parametrii funcției sunt inițializați cu valorile argumentelor.

▼ Exemplul 1

Definim o funcție care să calculeze (returneze) pentru un număr natural `n` dat suma primelor `n` numere naturale.

Matematic, funcția este definită pe mulțimea numerelor întregi cu valori în mulțimea numerelor întregi. suma: $\mathbb{Z} \rightarrow \mathbb{Z}$, $\text{suma}(n) = 1+2+3+\dots+n$

1. Cea mai simplă și eficientă variantă:

```

int suma(int n)
{
    return n*(n+1)/2
}

```

2. Varianta iterativă (repetitivă)

```

int suma(int n)
{
    int s = 0;
    int i;
    for(i=1; i<=n; ++i)
        s += i;
    return s;
}

```

3. Varianta recursivă

```

int suma(int n)
{
    if (n==0)
        return 0;
    else
        return n+suma(n-1);
}

```

▼ Exemplul 2 - Ce se întâmplă?

```

void swap(int x, int y){
    int temp = x; x = y; y = temp;
    cout << "x=" << x<< ",y=" << y<< "\n";
}
int main(void){
    int a = 2, b = 3;
    swap(a, b); // x = 3, y = 2
    cout << "a=" << a<< ",b=" << b<< "\n";
    // a = 2, b = 3
}

```

▼ Exemplul 3 - Ce se întâmplă?

```

void swap(int& x, int& y){
    int temp = x; x = y; y = temp;
    cout << "x=" << x<< ",y=" << y<< "\n";
}
int main(void){
    int a = 2, b = 3;
    swap(a, b); // x = 3, y = 2
    cout << "a=" << a<< ",b=" << b<< "\n";
}

```

```
// a = 3, b = 2
}
```

2.3.3. Transmiterea parametrilor

1. ▼ Variabile locale/globale/parametri

```
#include <iostream>
```

```
int x=10,t;
```

variabile globale, vizibile și în funcția **egale**

```
char egal(int x, int y)
```

```
{ int a=1; t=x+y+a;
```

```
return x==y?'D':'N'; }
```

variabilă locală

parametri formali ai funcției **egal**, vizibili doar în funcția **egal**

```
void main() {
```

```
int y=2,z=3; t=1;
```

```
cout<<t<<x<<y<<z;
```

variabile globale invizibile în funcția **egal**

```
if (egal(y,z))=='D' cout<<"Da"; else cout<<"NU";
```

```
cout<<t;
```

```
} // 1,10,2,3,Nu (deoarece y->x, adica 2->x, z->y, adica z->y și 2!=3, apoi t=6
```

2. ▼ Parametri transmiși prin valoare

```
#include <iostream>
```

```
void F(int x, int y)
```

parametri formali transmiși prin valoare

```
{
```

```
x=2; y=3; cout<<x<<y;
```

```
}
```

```
void main()
```

```
{
```

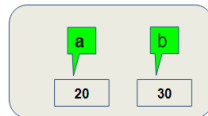
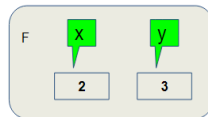
```
int a=20;
```

```
int b=30;
```

```
F(a,b);
```

```
cout<<a<<b; // 2, 3, 20, 30
```

```
}
```



3. ▼ Parametri transmiși prin referință

```
#include <iostream>
```

```
void F(int& x, int& y)
```

parametri formali transmiși prin referință

```
{
```

```
x=2; y=3; cout<<x<<y;
```

```
}
```

```
void main()
```

```
{
```

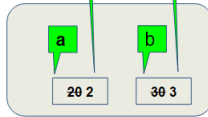
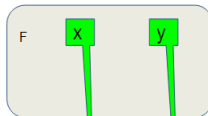
```
int a=20;
```

```
int b=30;
```

```
F(a,b);
```

```
cout<<a<<b; // 2, 3, 2, 3
```

```
}
```



4. ▼ Returnarea unei valori prin parametrul transmis prin referință - „proceduri”


```
#include <iostream>
void suma(int x, int y, int& suma)
{
    x=2; y=3; suma=x+y;
}
void main()
{
    int a=20; int b=30; int c;
    suma(2,3,c); cout<<c;
    suma(2,b,c); cout<<c;
    suma(2+a,b*3,c); cout<<c;
}
```

parametrii formali **transmiși prin valoare** sunt folosiți pentru *input*

parametrii formali **transmiși prin referință** sunt folosiți pentru *output*

modalități de apelare a funcției:
suma(expresie, expresie, variabila)
nu putem apela **suma(a,b,5)** sau **suma(a,b,5+c)**

5. ▼ Returnarea unei valori cu return „prin tipul funcției”

```
#include <iostream>
int suma(int x, int y)
{
    int suma;
    x=2; y=3; suma=x+y;
    return suma;
}
void main() {
    int a=20; int b=30; int c;
    cout<<suma(2,3);
    cout<<suma(2+b,a);
    c=suma(2+a,b-1); cout<<c;
}
```

parametrii formali **transmiși prin valoare** sunt folosiți pentru *input*

parametrii formali **transmiși prin referință** sunt folosiți pentru *output*

modalități de apelare a funcției **suma** în care valoarea returnată prin tipul funcției

2.3.4. Parametri implicați

```
#include<iostream>
using namespace std;
// 0 functie cu parametri implicați, poate fi apelata cu
// 2 argumente, 3 argumente sau 4 argumente.
int sum(int x, int y, int z=0, int w=0)
{
    return (x + y + z + w);
}
int main()
{
    cout << sum(10, 15) << endl;
    cout << sum(10, 15, 25) << endl;
    cout << sum(10, 15, 25, 30) << endl;
    return 0;
}
```

2.3.5. Funcții cu număr variabil de parametri

În C putem avea funcții cu număr variabil de parametri. Astfel de funcții vor avea mereu ultimul argument 3 puncte (...), iar argumentul din față sa va fi un *int*, care reprezintă numărul de argumente variabile setate la apelul funcției.

Sintaxa unei astfel de funcții și a unor apeluri:

```
int functie(int numar_argumente, ... ) >{
    .....
}
int main() {
    functie(3, 1, 2, 3);
    functie(4, 1, 2, 3, 4);
}
```

Folosirea funcțiilor cu număr variabil de parametri: Pentru a folosi funcții cu număr variabil de parametri, trebuie să includem fișierul header *stdarg.h*, care definește anumite macro-uri și funcții necesare. Pașii pentru a folosi o funcție cu număr variabil de parametri:

- definește o funcție care are penultimul argument un *int* (numărul de argumente primite la apel) și ultimul parametru "3 puncte" (...);
- crează o variabilă de tipul *va_list*;
- folosește macro-ul *va_start* și parametrul de *int* pentru a inițializa *va_list*;
- folosește macro-ul *va_arg* și variabila de tipul *va_list* pentru a accesa fiecare parametru din listă;
- folosește macro-ul *va_end* ca să cureți memoria alocată variabilei de tipul *va_list*.

▼ Exemplul 1 - sumă de numere

```
#include <stdio.h>
#include <stdarg.h>
int suma(int numar_parametri,...) {
    va_list lista_parametri;
    int suma_numere = 0;
    int i;
    va_start(lista_parametri, numar_parametri);
    for (i = 0; i < numar_parametri; i++) {
        suma_numere = suma_numere + va_arg(lista_parametri, int);
    }
    va_end(lista_parametri);
    return suma_numere;
}
int main() {
    printf("Suma numerelor 2, 3, 4, 5 = %d\n", suma(4, 2,3,4,5));
    printf("Suma numerelor 5, 10, 15 = %d\n", suma(3, 5,10,15));
}
```

▼ Exemplul 2 - concatenare de șiruri

Iată, în continuare, o funcție ce concatenează un număr arbitrar de șiruri de caractere.

```
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>

char *vstrcat(char *fmt, ...)
{
    size_t len;
    char *buf;
    va_list argp;
    char *p;

    if (fmt==NULL)
        return NULL;
    len = strlen(fmt);
    va_start(argp,fmt);

    while((p=va_arg(argp, char *)) != NULL)
        len+=strlen(p);

    va_end(argp);
    buf =(char *)malloc(len+1);
    if (buf==NULL)
        return NULL;

    strcpy(retbuf,fmt);
    va_start(argp,fmt);

    while((p = va_arg(argp, char *)) != NULL)
        strcat(buf,p);
    va_end(argp);
    return buf;
}
```

Modul de utilizare al acestei funcții:

```
char *sir = vstrcat("Ghici, cine ", "vine ", "la cina!",(char *)NULL);
```

Se observa cast-ul de la ultimul argument. Declarația `va_list argp;` definește lista de argumente ce va fi utilizată de macrourele `va_arg`, `va_end`. Înainte de a descărca argumentele din stivă este necesară inițializarea listei de argumente: `va_start(argp,fmt);` inițializează lista de argumente `argp` la primul argument fixat din lista variabilă (`fmt`). Rutina `va_arg` descarcă stiva de argumente. Ea primește doi parametri: numele listei de argumente și tipul datei care se descarcă.

2.3.6. Parametrii de tip pointer

Prezintă același comportament ca și parametrii de tip referință (se pasează adresa și nu valoarea) Exemplu de program cu parametrii trimiși prin valoare și parametrii trimiși prin referință:

```
#include <stdio.h>
#include <iostream>
void schimba_pointer(int *num1, int *num2)
{
    int tempnum;
    tempnum = *num1;
    *num1 = *num2;
    *num2 = tempnum;
}
void schimba_valoare(int num1, int num2)
{
    int tempnum;
    tempnum = num1;
    num1 = num2;
    num2 = tempnum;
}

int main( )
{
    int v1 = 10, v2 = 20;
    cout <<v1<<" "<<v2; //10 20
    cout<<"\n";
}
```

```
    schimba_valoare(v1,v2);  
    cout <<v1<<" "<<v2; //10 20  
    cout<<"\n";  
    schimba_pointer(&v1, &v2);  
    cout <<v1<<" "<<v2; //20 10  
    cout<<"\n";  
}
```
